The CUDA kernels of RutticeQCD, one by one

Wei-Lun Chen

November 14, 2026 (internal note)

For every kernel: what it computes, written out line by line, how its threads are laid out, and what in memory decides its speed. File names are under `cuda_bridge/include/`. The communication kernels are in the note on several GPUs.

Contents

1 How the fields lie in memory **2**
1.1 Blocks of 32 sites . . . 2
1.2 Spinors with a fifth dimension . . . 2
1.3 Links: two rows are enough . . . 3

2 The domain-wall operator **3**
2.1 What the operator is . . . 3
2.2 `fifth_term`: $a\psi_s + y(Y\psi)_s$ . . . 3
2.3 `hop_kernel`: out = [from−] $H(b - cY)$ in . . . 3
2.4 `mdwf_dslash_1pass_kernel`: all of D in one pass . . . 4
2.5 `mdwf_fifth_kernel`: $(a_1 + y_1 Y)x + (a_2 + y_2 Y)z$ . . . 4
2.6 The fifth dimension as a matrix: `fifth_gemm` . . . 4
2.7 `hop_gemm_kernel`: hops, then a matrix, in one pass . . . 5
2.8 Small kernels . . . 6

3 Wilson and clover **6**
3.1 The Wilson operator . . . 6
3.2 `clover_term_kernel`: $T\psi$ . . . 6
3.3 The clover operator . . . 6
3.4 `clover_blocks_kernel`: $T_\pm$, $T_\pm^{-1}$ and $\ln|\det T_\pm|$ from the links . . . 6
3.5 The force of the clover term: insertion, then leaves . . . 7

4 The fermion force: `hop_force_kernel` **8**

5 The gauge field **8**
5.1 Layout . . . 8
5.2 `stout_step_kernel`: $V = e^{iQ}U$ . . . 8
5.3 `stout_chain_local_kernel` and `stout_chain_staples_kernel` . . . 9
5.4 `gauge_force_kernel` and `gauge_traces_kernel` . . . 9

5.5 `links_exp_update_kernel`: $U \leftarrow e^{\epsilon H} U$ . . . 9
5.6 Other small kernels . . . 9

6 The multigrid: P **9**
6.1 What P holds . . . 9
6.2 `fold_restrict_kernel`: coarse $= P^\dagger$fine . . . 10
6.3 `fold_prolong_kernel`: fine $= P$coarse . . . 11
6.4 Orthonormalizing P . . . 11

7 The multigrid: the coarse operator **11**
7.1 Probing: `hop_faces_kernel`, `coarse_units`, `coarse_columns` . . . 11

8 The Schwarz smoother (SAP) on the device **12**
8.1 `coarse_pass_kernel` . . . 13
8.2 `blocks_inverse_kernel`: Gauss–Jordan . . . 13
8.3 `blocks_abs_inverse_gamma5_kernel`: $|H|^{-1}$ by Jacobi . . . 13
8.4 The exact adjoint: `coarse_adjoint_kernel` . . . 13
8.5 The adjoint without a copy: `coarse_adjoint_pass_kernel` . . . 13

9 The levels below the coarse lattice **14**

10 Linear algebra: `blas.cuh` **15**
10.1 Fused operations . . . 15
10.2 Deterministic reductions . . . 15
10.3 Sets of vectors . . . 15

1 How the fields lie in memory

1.1 Blocks of 32 sites

A field has N_c reals per site. Sites are grouped by 32, and inside a group the same component of the 32 sites is consecutive:

$$a(c,n) = \big(\lfloor n/32 \rfloor \cdot N_c + c\big) \cdot 32 + (n \bmod 32). \tag{1.1}$$

This layout matches a warp of 32 threads: if thread j handles site $32k+j$ and reads component c,

$$a(c, 32k+j) = (kN_c + c) \cdot 32 + j, \qquad j = 0, \ldots, 31, \tag{1.2}$$

32 consecutive numbers: one memory transaction. This is the only rule behind every layout below.

1.2 Spinors with a fifth dimension

A domain-wall spinor has L_s spinors of 24 reals per site. Component (color k, spin a, real/imag r) at fifth coordinate s is component

$$c = 24s + 2(k + 3a) + r \tag{1.3}$$

of the site, $N_c = 24L_s$. A field on one parity numbers its sites by $\lfloor x/2 \rfloor$:

$$n_{\text{half}} = \lfloor x/2 \rfloor + \tfrac{L_x}{2}\big(y + L_y(z + L_z t)\big), \qquad x = 2\lfloor x/2 \rfloor + \big((p+y+z+t) \bmod 2\big), \tag{1.4}$$

the second line recovering x from the parity p. Several vectors of a field lie one after another, each padded to a whole number of blocks.

1.3 Links: two rows are enough

A link $U \in SU(3)$ has rows r_0, r_1, r_2. Unitarity and $\det U = 1$ give

$$r_2 = \overline{r_0 \times r_1}, \tag{1.5}$$

$$(r_2)_c = \overline{(r_0)_a (r_1)_b - (r_0)_b (r_1)_a}, \qquad a = c+1,\ b = c+2 \pmod 3. \tag{1.6}$$

So the operators keep 12 reals per link instead of 18 when the links are in $SU(3)$, and rebuild row 2 in registers (`load_link` in `hops.cuh`). A hop reads mostly links, so this saves about a third of what it reads.

2 The domain-wall operator

2.1 What the operator is

The Möbius operator on ψ_s, $s = 0, \ldots, L_s - 1$, is

$$D\psi_s = (b\, D_W + 1)\psi_s + (c\, D_W - 1)\big(P_-\psi_{s+1} + P_+\psi_{s-1}\big), \tag{2.1}$$

with $D_W = (4 - M_5) - \frac{1}{2}\tilde{H}$ and $\tilde{H}$ the hops without κ. Define the fifth-dimension hop

$$(Y\psi)_s = -P_-\psi_{s+1} - P_+\psi_{s-1} \qquad (+m \text{ in place of } -1 \text{ across the walls}). \tag{2.2}$$

Then $P_-\psi_{s+1} + P_+\psi_{s-1} = -(Y\psi)_s$, and

$$D\psi = (bD_W + 1)\psi - (cD_W - 1)Y\psi \tag{2.3}$$

$$= \big[b(4 - M_5) + 1\big]\psi + \big[1 - c(4 - M_5)\big]Y\psi - \tfrac{1}{2}\tilde{H}\,(b\psi - cY\psi) \tag{2.4}$$

$$= \underbrace{(d_+ + d_- Y)\psi}_{C\psi} - \tfrac{1}{2}\tilde{H}\,(b - cY)\psi, \tag{2.5}$$

with $d_+ = 1 + b(4 - M_5)$ and $d_- = 1 - c(4 - M_5)$. C acts within a four-dimensional site; the hops act on $w = (b - cY)\psi$. Every kernel below is a piece of this line.

2.2 `fifth_term`: $a\psi_s + y(Y\psi)_s$

$$w_s = a\,\psi_s + y\,(Y\psi)_s \tag{2.6}$$

$$= a\,\psi_s - y\,P_-\psi_{s+1} - y\,P_+\psi_{s-1} \qquad (0 < s < L_s - 1), \tag{2.7}$$

$$w_0 = a\,\psi_0 - y\,P_-\psi_1 + y\,m\,P_+\psi_{L_s-1}, \tag{2.8}$$

$$w_{L_s-1} = a\,\psi_{L_s-1} + y\,m\,P_-\psi_0 - y\,P_+\psi_{L_s-2}. \tag{2.9}$$

For $Y^\dagger$ the projectors swap: P_+ from above, P_- from below.

2.3 `hop_kernel`: out = [from−] $H(b - cY)$ in

At output site x and fifth coordinate s:

$$\mathrm{out}_s(x) = -\kappa \sum_\mu \Big[(1 - s_\pm\gamma_\mu)U_\mu(x)\, w_s(x + \hat{\mu}) + (1 + s_\pm\gamma_\mu)U_\mu(x - \hat{\mu})^\dagger\, w_s(x - \hat{\mu})\Big], \tag{2.10}$$

with w_s from `fifth_term` computed on the fly at the neighbor ($a = b$, $y = -c$) and $s_\pm = \pm 1$ for H, $H^\dagger$. A leg is done in three steps (`leg_half`, link, `add_leg`). For $\mu = 0$ (Sec. 1.1 of the other note):

$$h_0 = w_0 + i s_\pm w_3, \qquad h_1 = w_1 + i s_\pm w_2, \tag{2.11}$$

$$t_a = U\, h_a \quad (a = 0, 1), \tag{2.12}$$

$$\text{out}_0 \mathrel{+}= -\kappa\, t_0, \qquad \text{out}_1 \mathrel{+}= -\kappa\, t_1, \tag{2.13}$$

$$\text{out}_3 \mathrel{+}= -\kappa\, (-i s_\pm)\, t_0, \qquad \text{out}_2 \mathrel{+}= -\kappa\, (-i s_\pm)\, t_1. \tag{2.14}$$

Two $SU(3)$ products per leg instead of four.

Threads. There is one thread for each (site, fiber); a fiber is one s of one vector. The threads of a block are 2D: `threadIdx.x` the site (consecutive sites: coalesced by Eq. (1.1)), `threadIdx.y` the fiber. All fibers of a site read the same eight links, which then come from the cache after the first. At most 256 threads per block: a thread may use 255 registers and a block has 65536.

Double precision. The two legs of a direction are done one after the other, so only one spinor and one link are live: 168 registers, three blocks per multiprocessor. The sums go into `out` in the same order either way.

From. With `from`, out = from − sum. `from` may be `out` itself, so the kernel reads all 24 reals of `from` into registers before it writes any: otherwise every load waits for the store before it (no `__restrict__` is possible), which cost the Wilson operator 10–15%.

The Wilson and clover operators are this kernel too, with two template parameters. `FourD`: $L_s = 1$, $b = 1$, $c = 0$ are set as constants, so the compiler removes the fifth dimension (the reads of $s \pm 1$ and the products with $c = 0$); with them known only at run time the Wilson operator took half as long again. The Wilson operator is

$$D\psi = \psi + H_\kappa \psi = \psi - H_{-\kappa}\psi, \tag{2.15}$$

`from` $= \psi$ at $-\kappa$: each leg changes sign exactly, so the result is the same to the last bit as $\psi + H_\kappa\psi$. `Clover`: the sum starts from T from (`clover_site`) and the legs are added to it, out $= T\psi + H\psi$ in one pass, the order of the old fused kernel. Because the interior and surface parts (Sec. 2 of the multi-GPU note) are instances of the same kernel, the two operators on several processes give the result of one bit for bit. On the GPU at 16^4 their time is that of the two kernels they replace (`wilson_interior_kernel`, `clover_dslash_kernel`).

2.4 `mdwf_dslash_1pass_kernel`: all of D in one pass

$$\text{out}_s(x) = \underbrace{d_+\psi_s(x) + d_-(Y\psi)_s(x)}_{\texttt{fifth_term}(a=d_+,\, y=d_-)} + \text{hops}(\kappa = \tfrac{1}{2}). \tag{2.16}$$

One read of ψ and the links, one write of out. Used for D on the whole lattice.

2.5 `mdwf_fifth_kernel`: $(a_1 + y_1 Y)x + (a_2 + y_2 Y)z$

Two `fifth_term`s added, for the parts of the even-odd operators that act within a site.

2.6 The fifth dimension as a matrix: `fifth_gemm`

In the Dirac representation γ_5 swaps the upper and lower spin pairs:

$$\gamma_5 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad P_\pm = \tfrac{1}{2}(1 \pm \gamma_5) = \tfrac{1}{2}\begin{pmatrix} 1 & \pm 1 \\ \pm 1 & 1 \end{pmatrix}. \tag{2.17}$$

Write a spinor at s as (upper u_s, lower l_s). Any operator of the form $\sum_{s'}(A_{ss'}P_+ + B_{ss'}P_-)$ gives

$$\mathrm{out}^u_s = \sum_{s'} \left[A_{ss'} \tfrac{u_{s'}+l_{s'}}{2} + B_{ss'} \tfrac{u_{s'}-l_{s'}}{2} \right] = \sum_{s'} \left[\tfrac{A+B}{2} u_{s'} + \tfrac{A-B}{2} l_{s'} \right]_{ss'}, \tag{2.18}$$

$$\mathrm{out}^l_s = \sum_{s'} \left[A_{ss'} \tfrac{u_{s'}+l_{s'}}{2} - B_{ss'} \tfrac{u_{s'}-l_{s'}}{2} \right] = \sum_{s'} \left[\tfrac{A-B}{2} u_{s'} + \tfrac{A+B}{2} l_{s'} \right]_{ss'}. \tag{2.19}$$

So the operator is one real $2L_s \times 2L_s$ matrix M acting on the rows $(u_0, l_0, u_1, l_1, \dots)$. Y, C, C^{-1}, $b - cY$, Neff's Q, their adjoints and their products are all of this form. Products are formed on the host, and one kernel applies the result. In a block of 32 sites the rows of the field are $2L_s$ rows of $12 \times 32 = 384$ reals:

$$\mathrm{out}\,(2L_s \times 384) = M\,(2L_s \times 2L_s)\ \mathrm{in}\,(2L_s \times 384). \tag{2.20}$$

Double precision on the tensor cores (`fifth_gemm_mma_kernel`). The instruction `mma.m8n8k4.f64` computes $D(8\times8)$ += $A(8\times4)\,B(4\times8)$; thread t of a warp holds $A[t/4][t \bmod 4]$, $B[t \bmod 4][t/4]$ and two entries of D. A warp takes a strip of 8 columns; for every 4 rows of the input it reads its 4×8 reals once and multiplies them with the tiles of M.

Why the XOR. M sits in shared memory, which has 32 banks of 4 bytes. A double occupies two banks, so double number k of a row of length K_{16} (a multiple of 16) is in banks

$$\mathrm{bank}(row, k) = 2(row \cdot K_{16} + k) \bmod 32 = 2k \bmod 32 \qquad (2K_{16} \equiv 0 \bmod 32). \tag{2.21}$$

A tile read takes rows 0..7 and columns $k_0..k_0+3$: all eight rows hit the same banks, an eight-way conflict. Store (row, k) at column $k' = k \oplus 4(row \bmod 4)$:

$$row \bmod 4 = 0 : k' \in \{k_0..k_0+3\}, \qquad row \bmod 4 = 1 : k' \in \{k_0..k_0+3\} \oplus 4, \tag{2.22}$$

$$row \bmod 4 = 2 : k' \in \{k_0..k_0+3\} \oplus 8, \qquad row \bmod 4 = 3 : k' \in \{k_0..k_0+3\} \oplus 12, \tag{2.23}$$

four disjoint groups of four doubles, $4 \times 4 \times 2 = 32$ banks. Rows r and $r+4$ still share, a two-way conflict. That is the least possible: 32 threads reading 8 bytes need 256 bytes, two passes of 128.

Single precision (`fifth_gemm_columns_kernel`): a thread per column of a block; it reads the $2L_s$ reals of its column once and keeps the $8N_T$ results in registers. All threads read the same entry of M at the same time, a broadcast. `fifth_gemm_tf32_kernel` does the product on the tensor cores in TF32 (10 bits of mantissa, relative error 10^{-3}), only for operators whose result is corrected afterwards (smoothers, inner solves).

2.7 `hop_gemm_kernel`: hops, then a matrix, in one pass

out = [from−] $M\,H(b - cY)$ in. The hops of a block of sites are written into shared memory as the $2L_s$ rows of the block, then multiplied by M as above. In double precision the `mma` reads 4 rows $j..j+3$ and 8 columns of the tile. A row is $12\times$(sites of the block) doubles; when that is a multiple of 16 doubles (128 bytes), rows j and $j+1$ hit the same banks. Storing column k of row j at $k \oplus 8(j \bmod 2)$ shifts the odd rows by 8 doubles = 16 banks:

$$\mathrm{bank}(j, k) = 2\big(k \oplus 8(j \bmod 2)\big) \bmod 32, \tag{2.24}$$

so even and odd rows use the two halves of the banks; rows j and $j+2$ share, two-way, again the least for 32 doubles. This is the XOR seen in the code; the face buffers of several GPUs need none (they are in global memory).

2.8 Small kernels

$$\texttt{gamma5}: \quad \mathrm{out}_s = \gamma_5\,\mathrm{in}_s, \text{ or with reflection } \mathrm{out}_s = \gamma_5\,\mathrm{in}_{L_s-1-s}\ (\Gamma_5 = \gamma_5 R), \tag{2.25}$$

$$\texttt{walls}(\to 4d): \quad q = P_-\psi_{s_-} + P_+\psi_{s_+}, \qquad (s_-, s_+) = (0, L_s-1) \text{ for the propagator}, \tag{2.26}$$

$$\texttt{walls}(\leftarrow 4d): \quad \psi_{s_-} = P_-\eta,\ \psi_{s_+} = P_+\eta, \tag{2.27}$$

$$\texttt{parity_copy}: \quad \mathrm{half}[n_{\mathrm{half}}] \leftrightarrow \mathrm{whole}[n]. \tag{2.28}$$

3 Wilson and clover

3.1 The Wilson operator

$D\psi = \psi + H\psi$: `hop_kernel` with `FourD` (Sec. 2.3). The fibers of a site are its vectors, which share the links.

3.2 `clover_term_kernel`: $T\psi$

$T = 1 - \kappa c_{SW}\sigma\cdot F$ commutes with γ_5. In the Dirac representation the eigenvectors of γ_5 are $(e_a \pm e_{a+2})/\sqrt{2}$, so in that basis T is two Hermitian 6×6 blocks (index color $+3\times$spin):

$$\psi^{\pm}_{k+3a} = \psi_{a,k} \pm \psi_{a+2,k}, \qquad a = 0,1 \quad (\text{the } 1/\sqrt{2} \text{ left out}), \tag{3.1}$$

$$\phi^{\pm} = T_{\pm}\,\psi^{\pm}, \tag{3.2}$$

$$(T\psi)_{a,k} = \tfrac{1}{2}(\phi^{+}_{k+3a} + \phi^{-}_{k+3a}), \qquad (T\psi)_{a+2,k} = \tfrac{1}{2}(\phi^{+}_{k+3a} - \phi^{-}_{k+3a}), \tag{3.3}$$

the two $1/\sqrt{2}$ making the $\frac{1}{2}$. A Hermitian 6×6 block is stored packed: 6 real diagonal entries and 15 complex above the diagonal,

$$6 + 2\cdot 15 = 36 \text{ reals}, \qquad 2 \text{ blocks} = 72 \text{ reals per site}. \tag{3.4}$$

The same kernel applies T^{-1} (its blocks made by `clover_blocks_kernel`, below).

3.3 The clover operator

$D\psi = T\psi + H\psi$ in one pass: `hop_kernel` with `Clover` (Sec. 2.3).

3.4 `clover_blocks_kernel`: $T_{\pm}$, $T_{\pm}^{-1}$ and $\ln|\det T_{\pm}|$ from the links

One thread for each pair (site, sign $s = \pm 1$); in double precision; the links extended one site deep (Sec. 5). First the field strength of each plane $\mu < \nu$ from the four leaves at x,

$$Q_{\mu\nu}(x) = \sum_{\ell=1}^{4} W_\ell(x), \qquad F_{\mu\nu}(x) = \tfrac{1}{8}(Q_{\mu\nu}(x) - Q_{\mu\nu}(x)^\dagger), \tag{3.5}$$

W_1 the path $x \to x+\hat\mu \to x+\hat\mu+\hat\nu \to x+\hat\nu \to x$ and W_2, W_3, W_4 the other three squares turning the same way (`leaves_of`). Then

$$T = 1 - \kappa c_{SW}\sum_{\mu<\nu} G_{\mu\nu}\otimes F_{\mu\nu}, \qquad G_{\mu\nu} = \gamma_\mu\gamma_\nu. \tag{3.6}$$

$G_{\mu\nu}$ has one entry in each row: $G_{r,c(r)} = g(r)$. From the tables of γ_μ,

$$c(r) = c_\nu(c_\mu(r)), \qquad g(r) = g_\mu(r)\,g_\nu(c_\mu(r)). \tag{3.7}$$

The block of sign s is T between the vectors $v_a = (e_a + s\,e_{a+2})/\sqrt{2}$, $a = 0, 1$:

$$(T_s)_{i+3a,\,j+3b} = \delta_{ab}\delta_{ij} - \kappa c_{SW} \sum_{\mu<\nu} h^s_{ab}(\mu\nu)\, F_{\mu\nu,ij}, \tag{3.8}$$

$$h^s_{ab} = v_a^\dagger G\, v_b = \tfrac{1}{2}\big(G_{ab} + s\,G_{a,b+2} + s\,G_{a+2,b} + G_{a+2,b+2}\big). \tag{3.9}$$

The kernel builds h^s by going over the four rows r of G: row r adds $g(r)$ to $h^s_{r \bmod 2,\, c(r) \bmod 2}$ with the factor $\frac{1}{2}\, s^{[r\geq 2]}\, s^{[c(r)\geq 2]}$.

T_s is packed (6 real diagonal entries, then the upper triangle) and inverted by Gauss–Jordan with partial pivoting, as `numerics::invert_complex`. Row operations that add a multiple of one row to another do not change the determinant, and scaling the pivot row by $1/p$ divides it by p, so

$$\ln|\det T_s| = \sum_{\text{columns}} \ln|p_{\text{col}}|. \tag{3.10}$$

The change of basis to the v_a is unitary, so $\ln|\det T(x)| = \ln|\det T_+| + \ln|\det T_-|$: the even-odd action adds them over the sites of a parity (on the host, in site order). A pivot below $6\,\epsilon \max|T_{rc}|$ marks the site singular, and `set_links` returns an error.

Registers: T_s and the inverse are 2×72 doubles, more than a thread keeps; the compiler puts part of them in local memory (L1). The kernel runs once a step of the integrator, next to a solve of hundreds of operator applications, so its speed does not matter.

3.5 The force of the clover term: insertion, then leaves

The clover term of D is $-\kappa c_{SW} \sum_{\mu<\nu} F_{\mu\nu} \otimes G_{\mu\nu}$. Its part of $-2\,\mathrm{Re}\,\zeta^\dagger \delta D \eta$, for $\delta U = XU$, is $\mathrm{Re}\,\mathrm{tr}(XM)$ with M made in two steps.

`clover_insertion_kernel`. For a pair (ζ, η) and a weight w, at every site and plane,

$$A_{\mu\nu,ij}(x) = \sum_s (G_{\mu\nu}\eta)_{s,i}(x)\, \overline{\zeta_{s,j}(x)} = \sum_s g(s)\, \eta_{c(s),i}(x)\, \overline{\zeta_{s,j}(x)}, \tag{3.11}$$

$$B_{\mu\nu}(x) \mathrel{+}= \frac{w\,\kappa c_{SW}}{4}\big(A_{\mu\nu}(x) - A_{\mu\nu}(x)^\dagger\big). \tag{3.12}$$

One thread a site; the 24 reals of each spinor read once. B of planes 0–3 sits in the four directions of one field of links and that of planes 4–5 in the first two of another, so the halo code of the links extends it.

`clover_log_det_insertion_kernel`. The derivative of $-2\ln|\det T_o|$ (the even-odd action) is the same insertion for the twelve pairs $(T^{-1}e_k, e_k)$ on the odd sites, e_k the unit vector of spin σ and color c. Only one component of e_k is nonzero, so the sum over k collapses:

$$A_{ij} = \sum_k \sum_s g(s)\, (e_k)_{c(s),i}\, \overline{(T^{-1}e_k)_{s,j}} = \sum_s g(s)\, \overline{(T^{-1})_{(s,j),(c(s),i)}}. \tag{3.13}$$

The entries of the 12×12 T^{-1} come from its packed blocks (`clover_entry`):

$$(T^{-1})_{(a,i),(b,j)} = \sum_{s=\pm} \tfrac{1}{2}\, s^{[a\geq 2]} s^{[b\geq 2]}\, (T_s^{-1})_{i+3(a \bmod 2),\, j+3(b \bmod 2)}. \tag{3.14}$$

`clover_leaves_kernel`. Once for all pairs: one thread a link (x, ρ). Every leaf of a plane containing ρ that passes the link contributes. Cut the leaf at the link: W_1 runs from the link to the leaf's corner, B of that corner is inserted, W_2 runs back,

$$M_\rho(x) = \sum_{\text{leaves}} \pm\, W_1\, B_{\mu\nu}(\text{corner})\, W_2, \tag{3.15}$$

+ if the leaf takes the link forward, − if backward (`hmc/fermion_action.rs`, `clover_leaves`). Because B is summed over the pairs first, a rational action with N poles walks the leaves once instead of N times; Bridge++ (`AForce_F_CloverTerm`) does the eight terms $R(1)\dots R(8)$ for every pair.

4 The fermion force: `hop_force_kernel`

For $\delta U_\mu(x) = X\,U_\mu(x)$ the change of $-2\,\mathrm{Re}\,\zeta^\dagger\,\delta D\,\eta$ is $\mathrm{Re\ tr}(X M_\mu(x))$ with

$$M_\mu(x) = -2c\sum_s\sum_{\text{spin}}\big[U_\mu(x)(1-\gamma_\mu)\eta_s(x+\hat\mu)\big]\zeta_s(x)^\dagger + 2c\sum_s\sum_{\text{spin}}\big[\gamma_5\eta_s(x)\big]\big[U_\mu(x)(1-\gamma_\mu)\gamma_5\zeta_s(x+\hat\mu)\big]^\dagger. \tag{4.1}$$

The kernel gets $U(1-\gamma_\mu)\eta(x+\hat\mu)$ from the forward leg of the hop kernel with $\kappa=-1$ (so the leg has coefficient $+1$), and the second bracket the same way from $\gamma_5\zeta$. Then the outer products, color by color:

$$(ab^\dagger)_{ij} = a_i\overline{b_j}, \qquad \mathrm{Re} = a_i^r b_j^r + a_i^i b_j^i, \quad \mathrm{Im} = a_i^i b_j^r - a_i^r b_j^i. \tag{4.2}$$

A thread per site, a loop over μ and s; M in the layout of the gauge fields (Sec. 5).

5 The gauge field

5.1 Layout

Links and fields of matrices (staples, Σ, forces) are whole 3×3 complex matrices, 18 reals per direction, 72 per site, in blocks of 32 sites (Eq. (1.1) with $N_c = 72$), all in double precision.

5.2 `stout_step_kernel`: $V = e^{iQ}U$

The staple sum and Q:

$$C_\mu(x) = \sum_{\nu\neq\mu}\rho_{\mu\nu}\Big[U_\nu(x)U_\mu(x+\hat\nu)U_\nu(x+\hat\mu)^\dagger + U_\nu(x-\hat\nu)^\dagger U_\mu(x-\hat\nu)U_\nu(x-\hat\nu+\hat\mu)\Big], \tag{5.1}$$

$$\Omega = C\,U^\dagger, \tag{5.2}$$

$$iQ = \tfrac12(\Omega-\Omega^\dagger) - \tfrac16\,\mathrm{tr}(\Omega-\Omega^\dagger)\cdot 1 \qquad \text{(traceless anti-Hermitian)}. \tag{5.3}$$

The exponential by Cayley–Hamilton. Q is Hermitian and traceless, so

$$Q^3 = c_1 Q + c_0\cdot 1, \qquad c_0 = \det Q = \tfrac13\,\mathrm{tr}\,Q^3, \qquad c_1 = \tfrac12\,\mathrm{tr}\,Q^2, \tag{5.4}$$

and every power of Q reduces to $1, Q, Q^2$:

$$e^{iQ} = f_0\cdot 1 + f_1 Q + f_2 Q^2. \tag{5.5}$$

With the eigenvalues parametrized as

$$\theta = \arccos\frac{c_0}{c_0^{\max}}, \qquad c_0^{\max} = 2\Big(\frac{c_1}{3}\Big)^{3/2}, \tag{5.6}$$

$$u = \sqrt{c_1/3}\,\cos\frac{\theta}{3}, \qquad w = \sqrt{c_1}\,\sin\frac{\theta}{3}, \tag{5.7}$$

Morningstar and Peardon find $f_j = h_j/(9u^2-w^2)$ with

$$h_0 = (u^2-w^2)e^{2iu} + e^{-iu}\big[8u^2\cos w + 2iu(3u^2+w^2)\,\xi_0(w)\big], \tag{5.8}$$

$$h_1 = 2u\,e^{2iu} - e^{-iu}\big[2u\cos w - i(3u^2-w^2)\,\xi_0(w)\big], \tag{5.9}$$

$$h_2 = e^{2iu} - e^{-iu}\big[\cos w + 3iu\,\xi_0(w)\big], \qquad \xi_0(w) = \frac{\sin w}{w}. \tag{5.10}$$

The kernel stores iQ and $(iQ)^2$ and uses $Q = -i(iQ)$, $Q^2 = -(iQ)^2$. When $|iQ|^2 \le 10^{-10}$ it takes $e^{iQ} = 1$. A thread for each (site, direction).

5.3 `stout_chain_local_kernel` and `stout_chain_staples_kernel`

The force on the thin links from Σ', the force on the smeared ones (Morningstar–Peardon eqs. (70)–(75)):

$$\Lambda = \tfrac{1}{2}(\Gamma + \Gamma^\dagger) - \tfrac{1}{6}\,\mathrm{tr}(\Gamma + \Gamma^\dagger), \tag{5.11}$$

$$\Gamma = \mathrm{tr}(\Sigma' B_1 U)\,Q + \mathrm{tr}(\Sigma' B_2 U)\,Q^2 + f_1\,U\Sigma' + f_2\,Q\,U\Sigma' + f_2\,U\Sigma' Q, \tag{5.12}$$

$$B_i = b_{i0} + b_{i1} Q + b_{i2} Q^2, \tag{5.13}$$

with b_{ij} derivatives of the f_j by c_0, c_1 (in the code, `r01`..`r22` and the `b`'s; $\xi_1(w) = (w\cos w - \sin w)/w^3$ by its series for $w < \frac{1}{4}$). Then

$$\Sigma = \Sigma'\,e^{iQ} + C^\dagger\,i\Lambda + \sum(\text{terms of the staples that contain } U_\mu(x)), \tag{5.14}$$

$$i\Theta = i\Lambda\,U. \tag{5.15}$$

The first kernel makes the first two terms and $i\Theta$; the second adds the six staple terms of each plane, which read $i\Theta$ at the neighbors. On several GPUs $i\Theta$ is extended between the two.

5.4 `gauge_force_kernel` and `gauge_traces_kernel`

$$F_\mu(x) = -\frac{\beta}{3}\,\mathrm{TA}\Big[U_\mu(x)\,\Sigma_\mu(x)^\dagger\Big], \tag{5.16}$$

$$\Sigma_\mu = c_{\mathrm{plaq}}\sum(\text{2 staples per plane}) + c_{\mathrm{rect}}\sum(\text{6 rectangle staples per plane}), \tag{5.17}$$

each staple a product of links along a path (`walk`). The traces kernel sums per site Re tr of the 6 plaquettes and 12 rectangles; the action follows on the host after the sum over sites.

5.5 `links_exp_update_kernel`: $U \leftarrow e^{\epsilon H} U$

The Taylor series to order n in nested (Horner) form:

$$e^{\epsilon H} U \approx \sum_{k=0}^{n} \frac{(\epsilon H)^k}{k!} U \tag{5.18}$$

$$= U + \epsilon H\Big(U + \frac{\epsilon H}{2}\Big(U + \frac{\epsilon H}{3}(\cdots)\Big)\Big), \tag{5.19}$$

computed from the inside out:

$$u^{(0)} = U, \qquad u^{(k+1)} = U + \frac{\epsilon}{n-k}\,H\,u^{(k)}, \qquad k = 0, \ldots, n-1. \tag{5.20}$$

Then reunitarized: $r_0 \to r_0/|r_0|$, $r_1 \to (r_1 - (\bar{r}_0 \cdot r_1) r_0)/|\cdot|$, $r_2 = \overline{r_0 \times r_1}$.

5.6 Other small kernels

`links_axpy` ($y \mathrel{+}= ax$), `links_ta` ($s\,\mathrm{TA}(m)$), `links_mul` (ab or $a^\dagger b$), `links_norm2` (per site $\sum$ reals^2, for $\frac{1}{2}\sum \mathrm{tr}\,P^2$).

6 The multigrid: P

6.1 What P holds

An aggregate keeps the whole fifth dimension and is split by the chirality of $\Gamma_5 = \gamma_5 R$, $(R\psi)_s = \psi_{L_s-1-s}$. For a test vector v:

$$v^{(c)} = \tfrac{1}{2}(1 + (-1)^c \Gamma_5) v, \qquad c = 0, 1. \tag{6.1}$$

At fifth coordinate s:

$$v_s^{(c)} = \tfrac{1}{2}\big(v_s + (-1)^c \gamma_5 v_{L_s-1-s}\big). \tag{6.2}$$

Now look at the mirror $L_s - 1 - s$:

$$\begin{aligned} v_{L_s-1-s}^{(c)} &= \tfrac{1}{2}\big(v_{L_s-1-s} + (-1)^c \gamma_5 v_s\big) && (6.3)\\ &= (-1)^c \gamma_5 \cdot \tfrac{1}{2}\big((-1)^c \gamma_5 v_{L_s-1-s} + v_s\big) \qquad (\gamma_5^2 = 1,\ ((-1)^c)^2 = 1) && (6.4)\\ &= (-1)^c \gamma_5\, v_s^{(c)}. && (6.5) \end{aligned}$$

So the half $s < L_s/2$ determines the rest, and P is stored folded: $12L_s$ reals per (site, test vector, chirality) instead of $24L_s$, in single or half precision. `fold_fill_kernel` writes $\frac{1}{2}(v_s \pm \gamma_5 v_{L_s-1-s})$ for $s < L_s/2$.

Four-dimensional fields (Wilson, clover): folded in spin. With $L_s = 1$ the chirality is that of γ_5 itself, and in the Dirac representation $\gamma_5 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ in 2×2 spin blocks. Write a spinor as its upper spins u (spins 0, 1) and lower spins l (spins 2, 3). Then

$$(1 + \sigma\gamma_5)\begin{pmatrix} u \\ l \end{pmatrix} = \begin{pmatrix} u + \sigma l \\ l + \sigma u \end{pmatrix} = \begin{pmatrix} w \\ \sigma w \end{pmatrix}, \qquad w = u + \sigma l, \quad \sigma = \pm 1, \tag{6.6}$$

because $l + \sigma u = \sigma(\sigma l + u)$ ($\sigma^2 = 1$). The lower spins of a vector of one chirality are σ times its upper ones, so the 12 reals of spins 0 and 1 are enough. The Gram–Schmidt of the CPU (`Basis::from_vectors`) treats every component alike, so the relation holds to the last bit; `DeviceBasis::new` keeps the upper spins and checks the lower ones. This is the folded storage with $L_s = 1$: the entries kept are those of $s = 0$ and spins $a = 0, 1$, and the mirror of (s, a) is $(L_s - 1 - s,\ \mathrm{col}[a]) = (0,\ a + 2)$ with γ_5's sign $+1$. The restriction of the next subsection becomes

$$\langle v^{(\sigma)}, f\rangle = \sum_{a<2} \overline{w_a}\, f_a + \sum_{a<2} \overline{\sigma w_a}\, f_{a+2} = \sum_{a<2} \overline{w_a}\,(f_a + \sigma f_{a+2}), \tag{6.7}$$

the same formula as the folding in s; so the kernels serve both, the loops running over the kept entries in their order (for L_s even: s, spin, color, the order they had).

6.2 `fold_restrict_kernel`: coarse $= P^\dagger$fine

For coarse site X, test vector i, chirality c:

$$\begin{aligned} \mathrm{coarse}(X, cn + i) &= \sum_{x \in X} \sum_{s=0}^{L_s-1} P(x, i, c)_s^\dagger\, f_s(x) && (6.8)\\ &= \sum_{x \in X} \sum_{s < L_s/2} \Big[P_s^\dagger f_s + P_{L_s-1-s}^\dagger f_{L_s-1-s}\Big] && (6.9)\\ &= \sum_{x \in X} \sum_{s < L_s/2} \Big[P_s^\dagger f_s + \big((-1)^c \gamma_5 P_s\big)^\dagger f_{L_s-1-s}\Big] && (6.10)\\ &= \sum_{x \in X} \sum_{s < L_s/2} P_s^\dagger \Big[f_s + (-1)^c \gamma_5 f_{L_s-1-s}\Big] \qquad (\gamma_5^\dagger = \gamma_5). && (6.11) \end{aligned}$$

The last line is what the kernel computes: one read of the folded P per pair $(s, L_s - 1 - s)$. A thread for each (coarse site, component), up to 8 vectors per read of P, which is most of the cost.

6.3 `fold_prolong_kernel`: fine $= P$coarse

$$f_s(x) = \sum_{c,i} P(x,i,c)_s \,\mathrm{coarse}(X, cn+i), \tag{6.12}$$

$$f_{L_s-1-s}(x) = \sum_{c,i} (-1)^c \gamma_5 P(x,i,c)_s \,\mathrm{coarse}(X, cn+i), \tag{6.13}$$

a thread for each (fine site, $s < L_s/2$, spin, color), writing s and its mirror.

6.4 Orthonormalizing P

Per (aggregate, chirality), the n vectors are made orthonormal over the aggregate. With $W = [w_1 \dots w_n]$:

$$G = W^\dagger W = LL^\dagger \text{ (Cholesky)}, \qquad Q = W\,(L^\dagger)^{-1} \;\Rightarrow\; Q^\dagger Q = L^{-1} G L^{-\dagger} = 1. \tag{6.14}$$

`fold_orthonormalise_cholesky_kernel` does this in double precision. Cholesky squares the condition number, so where a pivot is below 10^{-12} of its diagonal the kernel gives up on that site and `fold_orthonormalise_kernel` redoes it by modified Gram–Schmidt. The folded P holds half of every inner product, so the result is scaled by $1/\sqrt{2}$.

7 The multigrid: the coarse operator

7.1 Probing: `hop_faces_kernel`, `coarse_units`, `coarse_columns`

With e_j the unit vector of component j on every coarse site:

$$\phi_j = P\, e_j, \tag{7.1}$$

$$A(X)_{\cdot j} = [P^\dagger\, C\phi_j](X) + \sum_{\text{legs inside the block}} [P^\dagger \,\mathrm{leg}\, \phi_j](X), \tag{7.2}$$

$$B_d(X)_{\cdot j} = [P^\dagger \,\mathrm{leg}_d\, \phi_j](X) \quad \text{on the sites whose leg } d \text{ leaves the block.} \tag{7.3}$$

`hop_faces_kernel` computes the eight legs of a site one after the other and sorts each: a leg that comes from another block goes into the face of that leg, and a leg from the same block is added to `inside`. The face of leg d (direction μ, forward or backward) is the set of sites whose leg d comes from another block, $x_\mu \bmod b_\mu = b_\mu - 1$ forward and 0 backward, with b_μ the extent of a block in μ. Its sites form a lattice with extent L_μ/b_μ in μ and coordinate $\lfloor x_\mu/b_\mu \rfloor$ there, numbered like the lattice (`aggregate_face_site`); a field on it holds $\mathrm{leg}_d\,\phi_j$ where that is not zero. The restriction of the face is column j of B_d: in its leg mode the restriction kernel visits the sites of the face only and reads the field at their number on the face lattice. `coarse_columns` writes the result straight into the matrices, in the precision they are kept in. Several j at a time, as the vectors of the fields.

Each sum into `inside` is a rounded addition (`__fadd_rn`) in the order of the legs, forward $\mu = 0$ to 3, then backward. A launch for each leg rounded the same sums through memory, so the coarse operators are the same bit for bit (checksums of $P^\dagger D P$ and $P^\dagger D_{PV} P$ on $24^3 \times 48$). One pass reads ϕ and writes `inside` once and the faces once. With blocks of 4^4 the faces have 2 fields' worth of sites. A launch for each leg read and wrote `inside` eight times and wrote eight fields, each zero on 3/4 of its sites.

The sum lasts across the legs, and that sets the thread layout. A block is a warp of sites times 4 fibers (128 threads). Each thread keeps the 24 reals of its sum in shared memory, at a stride of the block, so a warp reads 32 consecutive words. Each leg calls `hops` with its mask known at compile time, through a `switch` on the leg. In single precision the kernel then runs four blocks an SM at

128 registers without spilling. Three other layouts failed on the GPU. The sum in registers spilled 448 bytes per thread at 128 registers. One block of 256 threads an SM, with no spill, was latency bound: 57 ms for two vectors on $24^3 \times 48$, $L_s = 8$. With the leg known only at run time, `hops` kept the code of all eight legs and spilled 384 bytes. The kernel takes 20 ms for two vectors. The eight launches of one leg each took 8×6.2 ms for one vector, so the pass is 4.9 times faster.

One function makes this for the three operators (`coarse::coarse_blocks`); an operator gives its part within a site, $C\phi_j$ above, and the coefficients of its hops: C and the hops of $b - cY$ at $\frac{1}{2}$ for domain-wall fermions, 1 and the hops at κ for Wilson fermions, the clover term T and the hops at κ for clover fermions.

8 The Schwarz smoother (SAP) on the device

SAP (`multigrid/sap.rs`) is written once; on the device the parts that touch the fields are three kernels and an instance of the hop kernel. The blocks are the aggregates; block X with block coordinates $(X_0, \ldots, X_3)$ on the whole lattice has color $\sum_\mu X_\mu \bmod 2$, so that blocks of one color do not touch. A cycle, for each color c:

$$e = 0, \qquad r \leftarrow r \text{ on the blocks of color } c,\ 0 \text{ elsewhere}, \tag{8.1}$$

$$\text{repeat } n \text{ times:} \quad p = D_c\, r, \quad \alpha_X = \frac{\langle p, r\rangle_X}{\langle p, p\rangle_X}, \quad e \mathrel{+}= \alpha_X r, \quad r \mathrel{-}= \alpha_X p, \tag{8.2}$$

$$x \mathrel{+}= e, \qquad r = b - Dx, \tag{8.3}$$

D_c being D without the legs that leave a block, on the blocks of color c, and $\langle\cdot,\cdot\rangle_X$ the inner product over the sites of block X (one vector at a time). α_X minimizes $|r - \alpha p|^2$ on the block:

$$\frac{\partial}{\partial\bar\alpha}\big(\langle r, r\rangle - \alpha\langle r, p\rangle - \bar\alpha\langle p, r\rangle + |\alpha|^2\langle p, p\rangle\big) = -\langle p, r\rangle + \alpha\langle p, p\rangle = 0. \tag{8.4}$$

The block operator: `hop_kernel` with `Cut`. A `BlockCut` gives the block extents b_μ, the color, and the parity of the block coordinates of the process's origin. At site x the forward leg in direction μ leaves the block when $x_\mu \bmod b_\mu = b_\mu - 1$, the backward one when $x_\mu \bmod b_\mu = 0$; those bits are taken out of the mask of legs (bit μ forward, bit $4+\mu$ backward), and a site of the other color writes zero. The rest is the kernel as it is: $\psi - H_{-\kappa}\psi$ for Wilson, $T\psi + H\psi$ for clover. `Cut` is a template parameter: with the mask known only at run time in every instance, the Wilson operator in single precision took 40% longer (the compiler no longer drops the tests of the eight legs).

`sap_alpha_kernel`. One CUDA block of 128 threads for each (block, vector); a block of the other color writes $\alpha = 0$. Thread t adds sites $t, t + 128, \ldots$ of the block in double precision, then a fixed tree over the threads in shared memory:

$$s_t \leftarrow s_t + s_{t+h}, \qquad h = 64, 32, \ldots, 1, \tag{8.5}$$

so the sum does not depend on the timing. Three sums at once: $\mathrm{Re}\langle p, r\rangle$, $\mathrm{Im}\langle p, r\rangle$, $\langle p, p\rangle$.

`sap_update_kernel`. A grid-stride loop over (site, vector): α of the site's block, then $e \mathrel{+}= \alpha r$ and $r \mathrel{-}= \alpha p$ in one pass, r read once (five reals read or written for each). `sap_keep_colour_kernel` zeroes the sites of the other color.

On several processes a block lies within a process, so D_c and the block sums need no exchange; only $r = b - Dx$ does, through the operator's halos. The device SAP agrees with the CPU's to 10^{-15} (Wilson, clover, one vector and three).

8.1 `coarse_pass_kernel`

One kernel for every part of the coarse operator:

$$\mathrm{out}(X) = M(X)\Big[\mathrm{base}(X) + A(X)\,d(X) \pm \sum_d B_d(X)\,h\big(N[d][X]\big)\Big], \tag{8.6}$$

each term optional. Choices give D_c, the Schur complement on the even sites $S = A_{ee} - B_{eo}A_{oo}^{-1}B_{oe}$, its source and solution, the block inverses. $D_c^\dagger = \Gamma_5 D_c \Gamma_5$ (Γ_5: -1 on the second half of the components) is the same kernel with a flag.

Threads. A block per (coarse site, vector); a warp per row; the lanes take pairs of columns $(2\ell, 2\ell+1), (2\ell+64, \dots)$, so a warp reads a row of a matrix as one stretch in 16-byte words. The fields at X and its 8 neighbors are read once into shared memory. The sum of a row is the lanes' parts added by a fixed butterfly, so the result does not depend on timing.

8.2 `blocks_inverse_kernel`: Gauss–Jordan

For each site, $[A \,|\, 1] \to [1 \,|\, A^{-1}]$, thread r holding row r:

$$\text{for } k = 0, \dots, \dim - 1: \tag{8.7}$$

$$p = \arg\max_{r \ge k} |a_{rk}|, \text{ swap rows } k, p, \tag{8.8}$$

$$\mathrm{row}_k \leftarrow \mathrm{row}_k / a_{kk}, \tag{8.9}$$

$$\mathrm{row}_r \leftarrow \mathrm{row}_r - a_{rk}\,\mathrm{row}_k \quad (r \ne k). \tag{8.10}$$

A zero pivot sets a failure flag (a singular block).

8.3 `blocks_abs_inverse_gamma5_kernel`: $|H|^{-1}$ by Jacobi

H = Hermitian part of $A\Gamma_5$. The cyclic Jacobi method: for each pair (p, q),

$$\text{phase: } h_{pq} = |h_{pq}|e^{i\varphi} \;\Rightarrow\; \text{multiply column } q \text{ by } e^{-i\varphi},\ \text{row } q \text{ by } e^{i\varphi}\ (h_{pq} \text{ now real}), \tag{8.11}$$

$$\text{rotation: } \tan 2\vartheta = \frac{2h_{pq}}{h_{qq} - h_{pp}},\ \text{rotate rows and columns } p, q \;\Rightarrow\; h_{pq} = 0, \tag{8.12}$$

repeated until the off-diagonal part vanishes, accumulating V. Then

$$|H|^{-1} = V\,|\Lambda|^{-1}V^\dagger, \tag{8.13}$$

with eigenvalues below a cutoff times the largest left out (they come from dependent vectors).

8.4 The exact adjoint: `coarse_adjoint_kernel`

$$(A^\dagger u)(Y) = A(Y)^\dagger u(Y) + \sum_d B_d\big(N[\bar{d}][Y]\big)^\dagger\, u\big(N[\bar{d}][Y]\big) \tag{8.14}$$

(derived in the other note); the kernel writes $B_d(X)^\dagger$ at the place of Y, entry by entry: $(m^\dagger)_{ij} = \overline{m_{ji}}$.

8.5 The adjoint without a copy: `coarse_adjoint_pass_kernel`

On one GPU the regularized solve of the Pauli-Villars factor applies $A^\dagger$ from the blocks of A, so that no second coarse operator is stored. Let $N[d][Y]$ be the neighbor of Y in direction d (eight directions $\pm\mu$), so that A is

$$(Av)(X) = A(X)\,v(X) + \sum_d B_d(X)\,v\big(N[d][X]\big). \tag{8.15}$$

The term $B_d(Y)\,v(N[d][Y])$ couples the output at Y to the input at $X = N[d][Y]$. Its adjoint therefore couples the output at X to the input at Y, with the matrix $B_d(Y)^\dagger$:

$$(A^\dagger u)(X) = A(X)^\dagger u(X) + \sum_d B_d(F[d][X])^\dagger\, u(F[d][X]), \qquad F[d][N[d][Y]] = Y. \tag{8.16}$$

$F[d][X]$ is the site whose hop d reaches X. On one process every site has exactly one such site for each d, so the table F is N inverted, one direction at a time; it is the only thing made for the pass (`adjoint_pass`). On a lattice cut between processes $F[d][X]$ can be a ghost whose links live on the neighboring process, and the copy of the previous subsection is used.

The matrix product. With $m = B_d(Y)$ and $u = u(Y)$, the entry c of $m^\dagger u$ is

$$(m^\dagger u)_c = \sum_k \overline{m_{kc}}\, u_k, \qquad \overline{(a+ib)}\,(x_r + ix_i) = (ax_r + bx_i) + i\,(ax_i - bx_r), \tag{8.17}$$

a sum over the rows k of m. The kernel reads m as `coarse_pass_kernel` does, a row at a time, which keeps the reads coalesced; only what is added changes: row k times the one number u_k goes into every column c.

Threads. A block per (coarse site X, vector). The fields at X and at the eight sites $F[d][X]$ are first read into shared memory ($9\times 2\,\mathrm{dim}$ reals). Warp w of W takes the rows $k = w, w+W, w+2W, \ldots$ of each of the nine matrices; lane ℓ holds the columns $c = 2\ell + 64i$, $i = 0, \ldots, 3$, in registers (16 reals), so $\mathrm{dim} \le 256$, that is `number_of_test_vectors` at most 128 (dim is twice the columns of P); above that the copy is used. All lanes of a warp read the same u_k from shared memory, a broadcast. After the nine matrices each warp writes its partial sums of all columns to shared memory ($W \times 2\,\mathrm{dim}$ reals), and thread c adds the W parts in the order $w = 0, 1, \ldots, W-1$. No atomics are used, so the result repeats bit for bit.

Order of the sum. The copy (`coarse_pass_kernel` on the stored $A^\dagger$) adds the terms of a row across the lanes by a butterfly; the pass adds them along the rows of a warp and then over the warps. The two give $A^\dagger$ to rounding but not bit for bit, so a run on one GPU and one on several (which uses the copy) differ by rounding in the coarse solve of the Pauli-Villars factor.

What limits it. Like the forward pass it reads the nine matrices of dim^2 complex numbers per site once, and is bound by memory. The final writes of the partial sums have a stride of two reals per lane: a two-way bank conflict in single precision, four-way in double, once per block.

9 The levels below the coarse lattice

`level_basis` : Q_{block} = Gram–Schmidt twice of the parts of the test vectors on a super-block and a half, (9.1)

`level_restrict` : $y = Q^\dagger v$ (a warp per output, lanes along the sum), (9.2)

`level_prolong` : $v = Qy$ or $v \mathrel{+}= Qy$, (9.3)

`level_sources` : $e_j = 1$ at component j of the sites of one color, (9.4)

`level_probe` : $A(t)_{\cdot j} = r_j(t)$ if t has the color, $B_d(t)_{\cdot j} = r_j(t)$ if $N[d][t]$ has it, (9.5)

`level_dense` : the last level as one $n \times n$ matrix, n = sites $\times$ components, (9.6)

`dense_apply` : out $= M\,r$. (9.7)

Probing works because the sources of one color are at least 3 apart in every direction, so

$$r_j(t) = \sum_{t'} D(t,t')e_j(t') = D(t,t'_*)\,\hat{e}_j \quad \text{with } t'_* \in \{t,\ t \pm \hat{\mu}\} \text{ the only source in reach.} \tag{9.8}$$

The response at t comes from one source only, and which one tells whether it is $A(t)$ or a $B_d(t)$. The dense inverse (LU) and $|H|^{-1}$ of the last level are done by cuSOLVER (`dense.cu`).

10 Linear algebra: `blas.cuh`

10.1 Fused operations

A solver step usually needs several updates and a norm at once, e.g.

$$r \leftarrow r - \alpha A p, \qquad |r|^2. \tag{10.1}$$

`blas_map_kernel` and `blas_reduce_kernel` take the operation as a functor (`Op`) and do it in one pass over the fields, every vector of a set at once.

10.2 Deterministic reductions

A sum over N numbers is done in two stages, both in a fixed order:

$$s_b = \sum_{p \in \text{block } b} x_p \quad \text{(a tree inside each CUDA block, in double precision)}, \tag{10.2}$$

$$S = \sum_b s_b \quad (\texttt{blas_finish_kernel},\ \text{one block, a fixed tree}). \tag{10.3}$$

No atomic additions, so the result does not depend on which block finishes first: runs repeat bit for bit. Only the final numbers go to the host.

10.3 Sets of vectors

`blas_gram_kernel`: all $\langle x_i, y_j \rangle$ in one pass. `blas_rotate_kernel`: $x \leftarrow xC$ in place for a small C. `blas_combine_kernel`: $\text{out}_j = \sum_i x_i c_{ij}$ for larger sets, the outputs of a chunk in registers.

Appendix: kernels removed

Extracted from Bridge++ with the rest but never called by the library, and removed on 2026-10-24 with the C tests that exercised only them: the Schwarz domain decomposition (`dd.cuh`), an older restriction, prolongation and coarse operator (`transfer.cuh`, `mdwf_transfer.cuh`, `coarse.cuh`, `coarse_eo.cuh`), multi-right-hand-side Wilson (`mrhs.cuh`), APE smearing and a plaquette (`gauge.cuh`), the Dirac/chiral conversion, a plain real `axpy`, and an unfinished Wilson face packing. Later the same day `wilson_interior_kernel` and `clover_dslash_kernel`, which `hop_kernel` replaced (Sec. 2.3) so that the Wilson and clover operators run on several processes.